TECHNICAL REPORT

Hierarchical ORAM: Communication, Cloud Cost, and Latency

17 October 2026 · Single projected AES per lookup

Result

At 10,000 queries/s, the seven-state same-AZ deployment costs **$6.69M/$9.97M annually** for $N = 2^{28}/2^{32}$ with $B = 1024$. At $B = 2048$, the cost is **$5.75M/$8.71M**, with about 241.0/239.2 ms mean and 362.6/358.3 ms p99 query latency.

Records	B	F	R	USD M/year	Mean ms	p99 ms	Max ms
2^{28}	1024	8	34	6.692	121.0	180.4	211.4
2^{32}	1024	10	46	9.967	121.8	180.4	213.9
2^{28}	2048	7	29	5.746	241.0	362.6	427.5
2^{32}	2048	9	39	8.706	239.2	358.3	427.4

Seven states. F and R are foreground and background workers per party; annual cost charges both parties. Max is a finite-trace observation.

These are newly sized and simulated deployments of the same construction: one address PRF per occupied table, private random dummy addresses, short-output AES pruning in both lookup and rebuilding, 64-entry lookup buckets, and Boolean fingerprints with packed payloads. All resource and price tables below use this protocol. There is no separate dummy PRF or dummy epoch key. The budget charges both parties and all seven stored states; each query runs against one ready state.

When a few hundred milliseconds are acceptable, $B = 2048$ is the cheaper tested choice. $B = 1024$ provides more latency margin. Rebuilding and prefetching remain the larger worker pool. The predictions assume that loading/transferring circuit material dominates CPU computation. They are conditional analytical results and finite deterministic traces, not AWS benchmarks or latency guarantees. All MB/GB/TB are decimal; MiB/GiB are binary.

For parties on different cloud platforms, Section 7 extends the analysis to private AWS–Google links and larger batches. The resulting budgets include cross-cloud bandwidth and allow a few seconds of query latency.

1 The current construction

1.1 Tables, reads, and representations

The branching factor is two. Logical table capacities are $n = B, 2B, \ldots, N$; each table has $2n$ main slots, grouped into $m_n = 2n/64$ lookup buckets, plus one global stash $G(n)$. The base is a hash table of capacity B, not a dense stash. A lower table is occupied half of a binary-counter cycle; the largest table is always occupied. With $L = \log_2(N/B)$, the mean number of occupied tables is $A = L/2 + 1$.

Hash each application key once to a secret 128-bit canonical fingerprint using a PRF. The numerical workload assumes a fixed 32-byte application key, charged as two AES-CMAC encryptions and its shared input [5]. A bitonic sort deduplicates the batch once; saved private comparison controls support final

result propagation and inverse routing. There is no recursive position map and no repetition of request deduplication at hierarchy levels.

Visit occupied tables from small to large. In each, reveal one selected bucket address and privately scan that bucket and the global stash. A match clears the entry's occupancy flag. Once a request has matched, later tables receive dummy probes. Reinsert one representative per deduplicated request, including a keyed absent tombstone when appropriate. The retained query fingerprint supplies the reinsertion key, so a second copy need not be extracted from the matching entry. Shared buckets and stashes receive one combined batch update, not conflicting per-query writes.

Each entry holds a 128-bit fingerprint and two Boolean flags (occupied and present), with one 16-byte label per bit per party. Its 256-bit payload is a 32-byte XOR share per party. This is an explicit word-OT representation; two ordinary GC labels do not encode an arbitrary 256-bit value. The persistent per-party entry size is

$$E = 16(128 + 2) + 32 = 2112 \text{ bytes.}$$

We use 32 bytes per half-gate AND and 160 bytes per packed conditional swap (80 in each direction). A Boolean conditional swap costs one AND per moved bit. Therefore a record swap costs

$$\underbrace{4096}_{\text{128-bit key}} + \underbrace{64}_{\text{two flags}} + \underbrace{160}_{\text{payload}} = 4320 \text{ bytes,}$$

plus 32 bytes per retained routing bit. The persistent fingerprint remains Boolean throughout this costed construction, including rebuilding.

Fingerprint equality uses 127 ANDs per scanned slot. Match, occupancy, active-request, and present handling have a 134-AND/slot allowance in the ledger, including equality. The allowance is conservative over a 130-AND core using one-hot XOR reductions. Packed extraction is charged separately at 160 bytes per candidate slot. Every lookup still scans a bucket and stash at every occupied table, including after a hit.

1.2 Rebuilding and recursive refinement

A carry rebuilds capacity n once per $2n$ requests; the largest table rebuilds once per N. Rebuilds are blocking for that state. Availability comes from other independent states, without a deamortized ORAM algorithm. The stages, including data movement, are:

1. Compact source main arrays. At the largest rebuild, discard absent tombstones. Append old stash entries and compact again to readmit them.
2. Retain a public n positions and compute fresh secret routing tags under the destination table's new epoch key. Dummy positions still receive fixed-shape work. Spread the compacted slots into half-full 512-slot buckets.
3. Run the binary 512-slot butterfly. A MergeSplit planner consumes only one routing bit per slot, but its secret controls move the complete entry and all unconsumed routing bits through ORCompact switches.
4. Refine each 512-slot parent into eight 64-slot leaves. Plan per-leaf membership, identify excess entries, collect them once into a 64-slot local stash, then recursively partition the feasible remainder. A public retained mask is inverse-routed to identify which original slots remain in each leaf.
5. Compact all local residual stashes into the table's global stash, with a fixed public capacity and a charged overflow check.

For a power-of-two q-slot ORCompact, the ledger uses $q \log_2 q/2$ swaps and a constructive planner allowance $q \log_2 q - q/2 - \log_2 q + 1$ ANDs. These are the project's cost derivation, not gate counts

quoted from the paper. The routing method follows the supplied Microrama and ORCompact sources [1, 2].

Global stash capacities are selected using a Chernoff/Poisson residual bound and rounded to multiples of eight. At $B = 1024$, the base stash is 56 slots and the largest stashes are 112/264 slots for $N = 2^{28}/2^{32}$. These capacities and the overflow checks are included in read and rebuild costs. Their existing failure accounting uses $Q = 2^{32}$ accesses; extending the failure budget to a full year remains a stated validation task, rather than an implicit annual guarantee.

2 One PRF and private dummy addresses

For every visit, each party samples a fresh private t_n-bit share, where $t_n = \log_2(2n/64)$. Inside the circuit, form $r = r_G \oplus r_E$ and reveal only

$$j = \mathrm{mux}(\mathrm{active}, H_{K_{n,e}}(x), r).$$

The dummy address is not opened separately. If both parties already knew it, comparing it with j would identify dummy visits. Generate and input shares for every visit, regardless of the hidden active flag. Our shared input allowance is $64t_n$ bytes: $48t_n$ forward and $16t_n$ reverse. The t_n-AND address mux is separately included in lookup logic.

The shares and their input labels can be prepared with the next circuit batch, so no extra dependent online coin-toss round is assumed. Prefetch accounts for both traffic directions and conservatively reserves the full input transcript in RAM. It does not treat that transcript as garbled gate bytes. The real-address PRFs can be computed before previous-table hit bits are available; only output selection depends on those bits.

The address PRF is full-round AES-128 followed by a fixed public output projection. Keep output bytes $(0, 13, 10, 7)$, concatenated in that order, and take the first t bits in a fixed bit order. These are zero-based byte positions in AES column-major serialization. Inverting the final ShiftRows maps the selected bytes to one round-9 column. For $t \le 32$, all 16 S-boxes in rounds 1–8, four in round 9, and $\lceil t/8 \rceil$ in round 10 suffice. Using the same 32-AND S-box as the full 5120-AND datapath gives the upper bound

$$g(t) = 32\,(128 + 4 + \lceil t/8 \rceil)\,.$$

MixColumns and AddRoundKey are linear. No AES round is removed and both lookup and rebuilding use the identical projection [3, 4]. We charge a complete S-box even for a partly used final byte. A completed full-output circuit would still cost 5120 ANDs; this saving requires pruning the unused computations.

Address bits	Retained S-boxes	ANDs	GC bytes
1–8	133	4256	136,192
9–16	134	4288	137,216
17–24	135	4320	138,240
25–32	136	4352	139,264

Expanded-key datapath; the full 1280-AND epoch-key schedule is charged separately.

Canonical fingerprints remain 128 bits. The two initial CMAC block encryptions remain full-output AES, and each table epoch receives a fresh independent secret address key with a full 1280-AND key

schedule and shared key input. There is no separate dummy key. For weights $w_n = 1/2$ at lower tables and $w_N = 1$, the amortized PRF datapath charge is

$$C_{\mathrm{PRF}} = 327680 + 64 \sum_n w_n g(t_n),$$

because the lookup weight and $n/\mathrm{rebuildInterval}(n)$ both equal w_n. Private dummy inputs add $64 \sum_n w_n t_n$ bytes/query. Epoch-key setup and inputs add $49152/B$ bytes/query: 48 at $B = 1024$, 24 at $B = 2048$.

Hash reuse already occurs within one table: one projected output supplies all butterfly and refinement address bits. Reusing address prefixes across tables or epochs can expose correlations and the hidden real-to-dummy transition; independent contexts are retained. Carrying an extra 128-bit Boolean hash through every swap costs another 4096 bytes/swap. One saved projected AES pays for only 33.25–34 such extra swaps. This makes that straightforward full-tag retention strategy unattractive, without ruling out a separately proved packed or selectively retained representation.

3 Communication breakdown and circuit interfaces

MB/query	28, 1024	32, 1024	28, 2048	32, 2048
Application-key shared input	0.016384	0.016384	0.016384	0.016384
Canonical fingerprint PRF	0.327680	0.327680	0.327680	0.327680
Deduplication and result propagation	0.459035	0.459035	0.549854	0.549854
Lookup: one address PRF	1.374208	1.652736	1.306112	1.584640
Lookup: private dummy randomness	0.009248	0.012640	0.009088	0.012480
Lookup: fingerprint equality	5.624576	7.981696	5.331968	7.689088
Lookup: other Boolean work	0.315920	0.447792	0.299648	0.431520
Lookup: packed value extraction	0.221440	0.314240	0.209920	0.302720
Lookup: address disclosure	0.000036	0.000049	0.000036	0.000049
Rebuild: Boolean field movement	8.691860	11.912748	8.528868	11.749756
Rebuild: fresh routing PRFs	1.374208	1.652736	1.306112	1.584640
Rebuild: planners and checks	0.507771	0.679557	0.496225	0.668012
Rebuild: epoch-key setup and inputs	0.000048	0.000048	0.000024	0.000024
Rebuild: packed value movement	0.309010	0.417390	0.302850	0.411230
Total useful communication	**19.231424**	**25.874732**	**18.684768**	**25.328076**

Both interparty directions, before framing and cache waste. Column labels are log2(records), batch size.

Rebuilding contributes 10.883/14.662 MB/query at $B = 1024$, more than half the useful total. Boolean fingerprint movement alone contributes 7.911/10.685 MB/query. Payload movement is small by comparison. The next table exposes the rebuild stages so that source compaction, recursive splitting, stash readmission, and residual collection are all visible.

MB/query	28, 1024	32, 1024	28, 2048	32, 2048
Source main-array compaction	1.032080	1.334512	1.007984	1.310416
Old-stash readmission	0.872112	1.157008	0.845824	1.130720
512-slot parent butterfly	5.897906	8.457143	5.850324	8.409561
Local refinement and residual extraction	1.610829	1.932994	1.530287	1.852453
Global residual collection	0.095586	0.127910	0.093396	0.125720
Fresh hashes and epoch-key setup	1.374256	1.652784	1.306136	1.584664
Tombstone and overflow checks	0.000129	0.000129	0.000128	0.000128

An alternative grouping of rebuilding from the previous table, not additional communication.

The cost requires compatible circuit composition: a common session-wide Free-XOR offset with odd color bit, unique gate/wire namespaces, and direct import of committed state-wire pairs. Immutable fingerprints and expanded keys may feed fresh circuits. Mutable occupancy outputs receive fresh wire pairs; changed values must never reuse the old pair. These are requirements on the eventual compiler and implementation.

Prefetched query ports also need fresh pairs. With common offset Δ, a translation from $W^b = W^0 \oplus b\Delta$ to V^b uses one public 16-byte value $T = W^0 \oplus V^0$. Reserve two 128-wire ports per occupied table, totaling 4096 bytes: one for the PRF/stash lane and one for the selected bucket equality lane. Cold stages without a bucket cache need at most the first port; retaining both is conservative. Every cached version and every query translation is single-use.

If instead each kernel independently re-encodes all persistent inputs, generic two-row unary bridges add approximately 5.849/8.279 MB/query at $B = 1024$, before further internal boundaries. The priced construction uses direct state composition; this overhead cannot simply be ignored by an implementation using unrelated labels. Cache binding and batch finalization remain compilation/security-composition TODOs.

4 Caching and the AWS resource bottleneck

4.1 Small-table RAM caches and lookahead

Cache the small tables near the batch-sized base, through capacity 2^{16}. Queue fresh immutable-fingerprint equality circuits and keep their state in RAM. Keep each table's global stash state and one future batch of its stash, PRF, and private-randomness material in RAM at every level. The remaining occupancy/present/active finalizer is generated dynamically.

A table of logical capacity n lasts n requests while present and has $2n/64$ buckets, so a bucket receives 32 accesses on average over its lifetime. That is not a useful per-batch upper bound. At the base with $B = 1024$, there are only 32 buckets and a mean of 32 accesses per bucket in that single batch. A four-version queue covers at most 12.5% of those requests. A larger adaptive queue is needed near the base.

With $X \sim \text{Binomial}(B, 1/m_n)$, choose the smallest queue width w satisfying $m_n \Pr[X > w]$ below the table's share of a 10^{-3} batch any-miss budget. At $B = 1024$, the seven widths are $[59, 37, 24, 17, 12, 9, 7]$. This is a performance budget under the ideal-PRF independent-address model, not an ORAM failure probability. Duplicates are removed before real probes; padded inactive requests get independent private dummy addresses.

GB per evaluator/state; $B = 1024$	Four versions	Adaptive
Bucket GC queue (134-AND allowance)	4.461167	11.354898
Persistent small-table state	0.550184	0.550184
One future stash batch	1.791492	1.791492
One future address-PRF batch	0.979370	0.979370
Private dummy-input transcripts	0.003670	0.003670
Total reserved RAM	7.785883	14.679615

All seven table capacities through 2^{16} reserved simultaneously; fresh single-use material.

The RAM table conservatively reserves the full 134-AND bucket allowance; the prefetched immutable equality component uses 127 ANDs/slot. Useful prefetched bytes remain in total communication. Add

only unused versions at table retirement and input bridges. Skip the final refill before a publicly known retirement. Cold buckets load state on demand and receive fresh circuits streamed to RAM; no cold-bucket GC disk queues are part of the priced deployment.

4.2 Network, EBS, and memory accounting

Use `c6in.32xlarge`, with 256 GiB RAM, 200 Gbps network per direction, and 100 Gbps aggregate EBS bandwidth [6, 7]. The modeled active rates per worker are

$$C_{\mathrm{NIC}} = 25 \cdot 0.65 \cdot 0.70 = 11.375 \text{ GB/s},$$
$$C_{\mathrm{EBS}} = 12.5 \cdot 0.65 = 8.125 \text{ GB/s},$$
$$C_{\mathrm{DRAM}} = 56 \text{ GB/s}.$$

The NIC factors are network and distributed efficiency. The DRAM rate is an explicit effective-loading proxy, not an AWS specification. Same-AZ pool sizing then limits modeled mean utilization to 70%; this factor is not applied again to active service durations. All traffic receives 5% framing.

Charge half the modeled external record read/write boundary volume as intra-party sends and receives in each party's limiting NIC direction. This assumes a distributed storage owner sends records to workers; gp3 is not assumed to support cross-host multi-attach. Actual gather/scatter may differ. If this internal volume is I per party per direction, both parties together send $2I$, not $4I$: counting each matching receive again would double-count network traffic.

MB/query	28, 1024	32, 1024	28, 2048	32, 2048
Unused equality-prefetch versions	0.412496	0.412496	0.410464	0.410464
Query-label bridges	0.040960	0.049152	0.038912	0.047104
Interparty total, 5% framing	20.669124	27.653199	20.090852	27.074927
Internal sends across both parties, unframed	6.885656	9.132376	6.766637	9.013357
All network sends, 5% framing	27.899063	37.242194	27.195820	36.538951
Limiting NIC direction per party, framed	23.996296	32.053382	23.364402	31.421488

If F is cross-party forward traffic, the limiting directional capacity uses $1.05(F + I)$, not the all-network sum. The reverse direction includes packed OTs and private-randomness input traffic and is checked separately. State I/O and DRAM loads are not extra 2PC communication, although remotely transferring a record is network traffic as represented by I.

Records	B	NIC-only hosts/party	Scheduled hosts/party	FG use	BG use
2^{28}	1024	31	42	68.1%	69.8%
2^{32}	1024	41	56	68.9%	69.8%
2^{28}	2048	30	36	67.6%	69.0%
2^{32}	2048	40	48	66.7%	70.0%

The NIC floor includes private dummy inputs, cache waste, bridges, and internal redistribution. Scheduled pools also include modeled loading/round delays.

Retain each 1024-slot MergeSplit and 512-slot refinement kernel in a small RAM tile. Materialize only the outer butterfly boundaries; source compactions, stash readmission, and global residual collection still incur their charged large-array passes. A conservative separate source/final boundary pass is also included. Keeping inner switches local is essential:

MB/query per party, before RAM state caching	$N = 2^{28}$	$N = 2^{32}$
Tiled rebuild and state I/O	10.974	15.130
Every swap layer spilled	23.566	32.561
Tiled + one GC disk read	29.645	40.240
Tiled + GC write/read	48.315	65.349

The complete state does not fit in the small-table cache: it is 2.268/36.284 TB per party per state at $B = 1024$. With tiled movement and fresh GC streaming, network transfer remains the main aggregate throughput limit. Counting two GC-memory passes, full switch-level record movement, and source/final boundary traffic, no rebuilt level in the tested cases requires more than 32.92 GB/s per worker to keep up with its modeled NIC/EBS service. This is below the 56 GB/s DRAM assumption. EBS or DRAM can still determine individual read-stage service. Spilling every switch layer or writing/rereading all GCs changes the bottleneck. Planner labels, OT matrices, state update amplification, and cache-owner distribution require implementation measurements.

5 Fleet scheduling and annual cost

5.1 Worker pools and independent states

There are F foreground and R background workers per party. Each pool jointly serves one job at a time. The foreground pool handles a batch; the background pool performs that state's rebuild and prepares its next circuit batch. Background jobs use preemptive shortest remaining work, allowing a small rebuild of another state to interrupt a large rebuild. A state stays unavailable until its whole rebuild and prefill finish. This changes scheduling across states, not the within-state ORAM algorithm.

States start warm at evenly spaced valid public counter offsets. Dispatch to the oldest ready state, queueing if none is available. The model uses a static lookup database: independent states contain the same logical data but evolve their own ORAM placements. Each request changes one state and is charged once. Application updates that must reach every state require an additional consistency protocol and are outside this workload.

For the same-AZ deployment, pool sizes are chosen separately so mean foreground and background occupancy, including modeled round/load waits, stay at or below 70%. The full binary-counter trace is then simulated. Each run spans two global N/B batch cycles: about 14.91 hours at $N = 2^{28}$ or 9.94 days at $N = 2^{32}$. The admission queue is included in the reported latency.

Seven states are used for the headline results. Five states are a useful smaller deployment comparison, using exactly the same current protocol and worker pools; they still exhibit occasional multi-second stalls in the larger-database traces. Additional states cost storage and cache capacity, not seven copies of the worker fleet:

Records	B	Five-state max ms	Seven-state max ms	Added USD M/year
2^{28}	1024	479.0	211.4	0.020
2^{32}	1024	5882.7	213.9	0.325
2^{28}	2048	525.9	427.5	0.020
2^{32}	2048	6073.1	427.4	0.325

Same current protocol and worker pools; only state, scratch, and cache count change. Finite traces, not tail guarantees.

The maxima are observations over two largest-table rebuild starts per trace. They are neither worst-case proofs nor annual tail guarantees. Cache misses, failures, bursty arrivals, and real checkpoint/suspension costs are not sampled. The preemption and storage models are assumptions to validate in a prototype.

5.2 AWS prices and storage reservations

Use the official us-east-1 price snapshot published 15 October 2026 and checked 17 October: Linux On-Demand `c6in.32xlarge` at $7.2576/hour; gp3 capacity $0.08/GiB-month; added throughput $0.04/(MiB/s)-month; and added IOPS $0.005/IOPS-month [9, 8]. The feed's throughput unit is GiB/s-month and is divided by 1024. Annualization uses 8760 instance hours and 12 storage-billing months.

Each worker receives eight gp3 volumes, each provisioned at 1491 MiB/s and 23,842 IOPS. This buys 12.507 GB/s aggregate raw throughput per worker, enough for the 12.5 GB/s modeled raw instance ceiling at 64 KiB operations. Each volume is at least 48 GiB for the IOPS/capacity constraint. The service model subsequently applies the 65% EBS efficiency. Paying only average throughput would not support the modeled active rebuild rate.

Reserve, per state and party, the persistent state P plus

$$S = 1.25\,[4N(2112 + 16t_N)]$$

bytes of scratch: one largest tagged source array and 25% working-space allowance. Reserving scratch for all seven states permits suspended rebuilds to retain work. This is a policy, not a proven maximum live-set bound. The per-party volumes are striped/distributed logically; internal record traffic is already charged rather than assuming direct shared disks.

For total workers $K_G + K_E = 2(F + R)$, volumes v, capacity D_v GiB, throughput T_v MiB/s, and IOPS J_v, the annual budget is

$$C_{\text{year}} = 63576.576(K_G + K_E) + \sum_v [0.96D_v + 0.48(T_v - 125)_+ + 0.06(J_v - 3000)_+].$$

The baseline places both parties at private addresses in the same Availability Zone, so this transfer placement has no separate EC2 regional data-transfer charge [10]. Cross-AZ placement would need a different budget.

Records	B	EC2	EBS capacity	EBS I/O	Total
2^{28}	1024	5.340	0.070	1.281	6.692
2^{32}	1024	7.121	1.139	1.708	9.967
2^{28}	2048	4.578	0.070	1.098	5.746
2^{32}	2048	6.103	1.138	1.464	8.706

Annual USD millions; both parties; seven states, scratch, and peak provisioned I/O included.

Records	B	All-state cache GB	Usable fleet RAM GB	EBS TB/party
2^{28}	1024	144.475	8658.7	39.174
2^{32}	1024	171.632	11544.9	636.411
2^{28}	2048	229.983	7421.7	39.174
2^{32}	2048	284.287	9895.6	636.411

EBS includes seven persistent states and seven scratch reservations. RAM is aggregate; cache placement and working buffers still need implementation.

The annualization is steady-state at 10,000 queries/s, or 315.36 billion queries/year. Initial database construction, cache warmup, client/application transport, application computation, taxes, and replica-update consistency are excluded. This is a semi-honest steady-state ledger; base-OT setup and malicious-security checks are outside its scope. CPU is excluded by the requested assumption. Each billed fleet must still meet the advertised NIC/EBS rates in the actual placement; both NIC cards and multiple flows are required for the modeled aggregate network rate [7].

6 Query latency and batch size

A central batcher receives 10,000 requests/s. For deterministic arrivals, the mean batch-fill delay is $(B-1)/(2q)$: 51.15 ms at $B = 1024$, 102.35 ms at $B = 2048$. It is not multiplied by the number of states. Splitting arrivals into seven independent batchers would give a different fill delay.

At each dependent table stage, service is the maximum of NIC transfer, EBS loading (including one 0.5 ms fixed wave allowance when needed), and DRAM loading. Those per-stage maxima are summed. Taking one maximum over the entire hierarchy would hide a level-local storage bottleneck. The state/stash reads within a table are assumed parallel.

Read packed-value selection can wait until all metadata reads finish and run as one batched OT layer. A parallel segmented propagation tree uses at most $2\log_2 B + 1$ packed layers; inverse bitonic routing uses $\log_2 B(\log_2 B + 1)/2$. With $k = \log_2 B$, the model's mean RTT-equivalent dependency allowance is

$$U = 2 + A + (2k + 1) + k(k + 1)/2.$$

Each unit costs 0.2 ms in the primary scenario. This is an explicit scheduling allowance, not a proof of the exact compiled protocol's round count. Preprocessed OT does not remove dependent online packed products. The reported query latencies include fill, active service, and the finite trace's admission queue.

Records	B	Primary ready	EBS 1 ms	EBS 3 ms	RTT 1 ms
2^{28}	1024	120.8	123.2	136.2	191.2
2^{32}	1024	121.7	125.0	140.3	193.7
2^{28}	2048	240.7	240.7	250.8	321.1
2^{32}	2048	239.0	239.0	251.7	321.0

Mean query ms with a ready state and unchanged foreground pool. Change one delay at a time.

The effect of changing batch size is visible directly in the communication ledger. Smaller batches save on the one-time bitonic work but add hierarchy levels, rebuild frequency, and a larger fraction of fixed round/load delay.

Records	B	Frontend	Lookup	Rebuild	Total
2^{28}	512	0.721	7.898	11.103	19.722
2^{28}	1024	0.803	7.545	10.883	19.231
2^{28}	2048	0.894	7.157	10.634	18.685
2^{32}	512	0.721	10.780	14.883	26.383
2^{32}	1024	0.803	10.409	14.662	25.875
2^{32}	2048	0.894	10.020	14.414	25.328

Useful MB/query, both directions; the same single-PRF construction at each batch size.

Reducing B from 1024 to 512 saves only about 0.083 MB/query in the frontend; total useful communication increases. In the resized five-state tests, $B = 512$ needs 59/78 workers per party and costs \$9.35M/\$13.11M annually, with mean latency about 60–61 ms. It is not a cost-saving choice for the stated tolerance. The headline seven-state $B = 2048$ deployments save about 13–14% versus $B = 1024$ and retain sub-430 ms observed maxima in these traces. Smaller batches remain relevant if a substantially tighter latency target is imposed.

7 Cross-cloud deployment

Placing the parties on AWS and Google Cloud costs **$68.6M/$95.9M annually** for $N = 2^{28}/2^{32}$ at 10,000 reads/s. With $B = 16{,}384$ and a 5 ms round-trip time, mean latency is about 2.3 seconds and p99 about 3.2 seconds. Cross-cloud networking accounts for 89–90% of the budget.

AWS us-east-1 hosts the garbler and Google us-east4 hosts the evaluator, connected privately in Northern Virginia. The workload remains a static database with 32-byte keys and 32-byte values. Both scenarios retain the construction, cache policy, and seven states; their RTTs are assumed.

RTT ms	Records	B	Mean s	p99 s	Infra.	Network	Total
5	2^{28}	16,384	2.27	3.20	6.5	62.1	**68.6**
5	2^{32}	16,384	2.24	3.15	10.1	85.8	**95.9**
10	2^{28}	32,768	4.57	6.38	6.8	64.9	**71.8**
10	2^{32}	32,768	4.54	6.33	10.8	91.5	**102.3**

Model estimates at 10,000 reads/s, seven states. Costs are annual USD millions for both parties. Infrastructure uses AWS-equivalent compute, storage, and I/O prices for both clouds; latency includes batch fill and admission queues.

7.1 Larger batches absorb round trips

At $B = 2048$, about 100 RTT-equivalent dependencies occupy the foreground pool for half a second at 5 ms RTT, while a batch arrives every 0.205 seconds. Relaxing response latency alone cannot make this pool sustain 10,000 reads/s. More workers cannot remove the dependency delay. Keeping $B = 2048$ would require concurrent batch pipelines and a revised scheduler.

Larger batches give the pools time to finish each job. At $B = 16{,}384$, the arrival interval is 1.638 seconds and mean fill delay is 0.819 seconds, supporting the 5 ms RTT scenario. At 10 ms RTT, $B = 32{,}768$ gives a 3.277-second interval and 1.638-second mean fill delay. Removing lower hierarchy levels also reduces communication per read despite the larger frontend sort.

Cross-cloud pools target at most 90% mean occupancy, including round and loading waits, versus 70% for same-AZ pools. This uses the extra latency budget to reduce fleet and burst-bandwidth requirements. Two global counter cycles include two largest-table rebuild starts per scenario; reported latencies include the admission queue.

7.2 Charge traffic in its actual direction

About 98.6% of cross-party bytes travel from garbler to evaluator. AWS flat-rate Direct Connect covers that dominant direction; only the smaller Google-to-AWS stream incurs a per-GiB transfer charge. Charge each send once, excluding matching receives and same-party transfers.

Records	B	AWS to Google	Google to AWS	Total MB/read	Forward Tbps
2^{28}	2,048	19.812	0.279	20.091	1.585
2^{32}	2,048	26.689	0.385	27.075	2.135
2^{28}	16,384	17.931	0.251	18.182	1.434
2^{32}	16,384	24.735	0.356	25.091	1.979
2^{28}	32,768	17.258	0.241	17.499	1.381
2^{32}	32,768	24.062	0.346	24.408	1.925

Directional columns are MB/read, including 5% framing, cache waste, and bridges. Forward throughput assumes 10,000 reads/s. Intra-party transfers are excluded.

7.3 Private-link prices and capacity

The price snapshot checked on 17 October 2026 combines an AWS Tier 1 100 Gbps Direct Connect port-pair at $102.74/hour with two Google 100 Gbps Cross-Cloud Interconnect circuits at $30/hour each and two 100 Gbps VLAN attachments at $1/hour each [11, 12]. Thus one redundant bundle costs

$$102.74 + 2(30 + 1) = \$164.74/\text{hour},$$

or $1.443M/year. Its usable capacity is 100 Gbps, including a redundant path; the two physical connections do not provide 200 Gbps of usable capacity. AWS outbound transfer is included for the covered path. Google charges $0.02/GiB for outbound transfer on the selected US path and nothing for inbound transfer. The selected topology connects primary and redundant ports on both providers [13].

Average traffic alone understates the capacity needed by the scheduled rebuild bursts. We reserve enough cross-cloud bandwidth for the foreground and background pools to use their full modeled active NIC rates simultaneously. For $F + R$ workers per party and H usable 100 Gbps bundles, the reservation satisfies

$$0.90\,H\,(12.5\text{ GB/s}) \;\geq (F + R)(11.375\text{ GB/s}).$$

The 10% link allowance is separate from worker-pool occupancy. A second check keeps mean traffic below 70% of raw link capacity; the burst condition determines the link counts in every scenario below.

RTT ms	Records	F	R	100 Gbps bundles	Links/year	Reverse/year
5	2^{28}	9	32	42	60.611	1.473
5	2^{32}	12	45	58	83.701	2.091
10	2^{28}	10	33	44	63.497	1.416
10	2^{32}	13	48	62	89.474	2.034

Workers are per party. Link and reverse-transfer costs are annual USD millions. Each bundle includes primary and redundant connections with 100 Gbps usable capacity in total.

This reservation covers even the portion of worker NIC capacity used for internal transfers, so cross-cloud bandwidth cannot further constrain the modeled schedule. Those internal bytes do not enter the billed reverse egress volume. With $q = 10{,}000$ reads/s and $d_{\leftarrow}$ reverse bytes/read, the annual networking cost is

$$C_{\text{net}} = 8760H(164.74) + \frac{(365 \cdot 24 \cdot 3600)q\,d_{\leftarrow}}{2^{30}}(0.02).$$

Network charges dominate the total, so smoothing rebuild traffic across time and sharing link capacity more tightly are useful optimization targets. The reported budgets retain the full simultaneous-burst reservation. A budget based only on average traffic would need a new schedule before inheriting these latency estimates.

Scope of the estimate. Compute, storage, and I/O for both parties use the existing AWS prices as a common infrastructure proxy. The CPU and storage assumptions are the same as in the baseline. Provider selection must confirm the assumed RTT, multi-terabit capacity, and eligibility of the chosen dedicated AWS port-pair for flat-rate billing. Google workers and their VLAN attachments must be in the same region to avoid extra regional transfer charges. Any separately charged installation, cross-connect, colocation, or optional network-service fees are outside this budget. The estimate is a deployment projection from component prices, not a negotiated quote.

8 Validation and remaining work

All current resource paths consume the same materialized single-PRF case. The annual model and report renderer reject a schedule stamped with a different address protocol, preventing old traces from silently entering new tables. Thirty-five tests, including six ORCompact/refinement checks, cover direction conservation, private dummy input prefetch in both directions, cache lifecycle, stagewise latency, counter carries, headroom, price units, storage/state charging, label translations, complete entry movement, and stale-schedule rejection.

The pruned AES computations match 32 independent native-AES key/plaintext vectors for one through four projected bytes and ordinary prefixes, plus a full AES known-answer test. The 32-AND S-box cost is held fixed so the projection saving is isolated; newer S-box circuits are not silently substituted. Functional pruning and ledger tests are not a compiled GC benchmark or a security proof for the full composition.

The outstanding tasks are concrete: compile and validate the label/cache interfaces and shared-bucket finalizer; implement the global planner's external-memory schedule; measure directional NIC throughput, effective DRAM loads, and EBS IOPS/waves; test real background preemption and bursty arrivals; and extend the overflow/fingerprint failure accounting from 2^{32} accesses to the annual workload. A year here is about 73.4 times that horizon. The current numerical capacities must not be presented as already proving an annual failure target.

Packing fingerprints only while moving them is a possible further optimization, but its conversion/composition kernels and new I/O schedule are not implemented. It is not included in any table in this report. The priced construction uses Boolean fingerprints and packed values consistently throughout.

A Reproduction

From the repository root, with Python, NumPy, and SciPy available:

```
python -m mpc_cost.hierarchical_throughput
python -m mpc_cost.hierarchical_communication_audit
python -m mpc_cost.hierarchical_prf_optimization
python -m mpc_cost.hierarchical_schedule
python -m mpc_cost.hierarchical_schedule --suite variants
python -m mpc_cost.hierarchical_annual
python -m mpc_cost.hierarchical_cross_cloud
python -m unittest discover -s mpc_cost -t . -p "test_hierarchical*.py"
python -m mpc_cost.render_hierarchical_throughput_report
```

The scheduling commands rerun the full deterministic traces rather than scaling an old price or latency by a byte-saving percentage. Current case construction is in `hierarchical_address.py`; decomposition, trace, and price results are stored separately under `mpc_cost/results`. The original split ledger remains a reproducible input to the explicit current-case transformation. It is not the protocol used for the final resource tables. The standalone LaTeX includes all numerical tables and needs no project-specific inputs to compile.

The cross-cloud command reruns the four larger-batch schedules and saves their traffic, resource counts, link reservations, and costs in `hierarchical_cross_cloud.json`. Published link prices and their source URLs are recorded in `mpc_cost/data/cross_cloud_prices_2026-10-17.json`.

References

[1] Microrama. User-supplied ongoing paper and the project's Binary Hierarchical ORAM for Interactive Two-Party Computation construction ledger, 17 October 2026.

[2] IACR ePrint 2022/1333. User-supplied ORCompact source, Sections 3, 5.1, and Appendix B. https://eprint.iacr.org/2022/1333

[3] NIST. FIPS 197, Advanced Encryption Standard (AES), updated May 2023. Sections 3.4 and 5.1. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf

[4] NIST. Circuit Complexity: List of Circuits. AES circuit and S-box gate counts. Accessed 17 October 2026. https://csrc.nist.gov/projects/circuit-complexity/list-of-circuits

[5] NIST. SP 800-38B: The CMAC Mode for Authentication. https://csrc.nist.gov/pubs/sp/800/38/b/upd1/final

[6] AWS. Specifications for EC2 compute optimized instances. https://docs.aws.amazon.com/ec2/latest/instancetypes/co.html

[7] AWS. EC2 instance network bandwidth. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-network-bandwidth.html

[8] AWS. EBS General Purpose SSD volumes. https://docs.aws.amazon.com/ebs/latest/userguide/general-purpose.html

[9] AWS. AmazonEC2 us-east-1 public price list, version 20261015191316, published 15 October 2026. Selected rows and SKU identifiers saved in `aws_hierarchical_prices_2026-10-17.json`. https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/current/us-east-1/index.csv

[10] AWS. Billing and metering for VPC owners and participants. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-share-billing.html

[11] AWS. Direct Connect flat-rate pricing. Tier 1 100 Gbps port-pair, accessed 17 October 2026. https://aws.amazon.com/directconnect/pricing/flat-rate/

[12] Google Cloud. Cloud Interconnect pricing, Cross-Cloud Interconnect section. Accessed 17 October 2026. https://cloud.google.com/network-connectivity/interconnect/pricing

[13] Google Cloud. Cross-Cloud Interconnect overview. https://docs.cloud.google.com/network-connectivity/docs/interconnect/concepts/cci-overview